

数据结构强化课考试

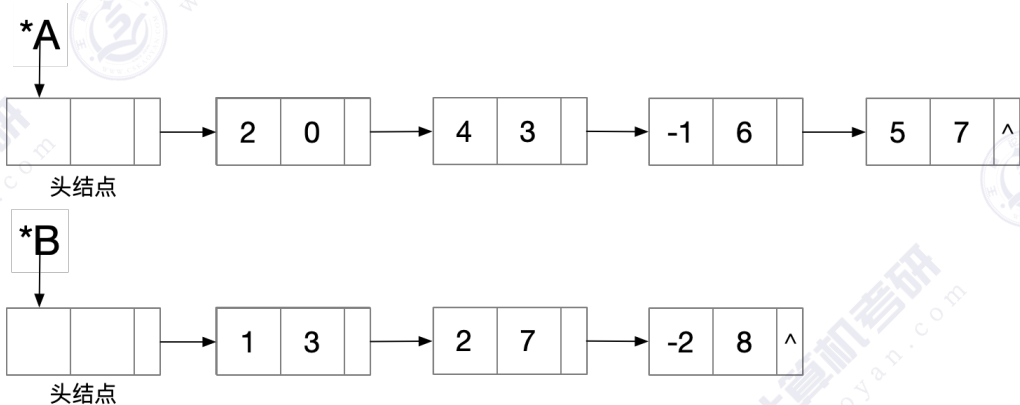
注：本试卷的个别题目难度略高于真题。做题时间不限，遇到不会的题目可以开卷考。

算法题

一、（13分）数学上的一元多项式 $P(x) = p_0 x^0 + p_1 x^1 + p_2 x^2 + \dots + p_n x^n$ ，可以用一个单链表来存储，结点的数据结构定义如下：

```
1 typedef struct Node {
2     int co;    //系数
3     int ex;    //指数
4     struct Node *next; //指向下一个结点
5 } Node, *Duoxiangshi;
```

其中，co 表示每一项的“系数”，ex 表示每一项的“指数”，next 为指向下一个结点的指针。我们规定：“系数”为0的项无需存储；各个项在链表中按“指数”递增存放；单链表有头结点。例如，多项式 $A(x) = 2 + 4x^3 - x^6 + 5x^7$ 和 多项式 $B(x) = x^3 + 2x^7 - 2x^8$ 可表示为：



现要求设计一个尽可能高效的算法，实现两个多项式的加法，并返回相加之后的结果，相加结果存储到多项式A。函数原型为：Duoxiangshi add(Duoxiangshi A, Duoxiangshi B)。要求：

- 1) 给出算法的基本设计思想。（4分）
- 2) 根据设计思想，采用C或C++语言描述算法，关键之处给出注释。（7分）
- 3) 说明你所设计算法的时间复杂度和空间复杂度。（2分）

【参考答案】

(1) 可以通过两个指针 `pA` 和 `pB` 同时遍历两个有序链表表示的多项式A、B，逐项比较指数并执行以下操作：

- 若 `pA` 和 `pB` 指数相等：将两个节点的系数相加，删除 `pB` 节点，继续遍历。
- 若 `pB` 指数较小：将 `pB` 插入到 `pA` 前面，并更新链表指针。
- 若 `pB` 指数较大：什么也不做，继续遍历 `pA`，直到找到下一个指数更大的 `pA` 结点。

若链表A遍历完后，链表B还有剩余结点没有合并到链表A，则直接将链表B剩余部分连接到链表A的末尾。

(2)

```
1 Duoxiangshi add(Duoxiangshi A, Duoxiangshi B){
2     Node * pA = A->next;
3     Node * pPreA = A;      //定义两个前后指针，用于遍历多项式A
4
5     Node * pB = B->next;
6
7     while (pA != NULL && pB != NULL){
8         if (pA->ex == pB->ex){
9             pA->co += pB->co;      //若指数相等，则系数相加
10            B->next = pB->next;    //把链表B的结点拆掉，并释放空间
11            free(pB);
12            pB = B->next;
13            pA = pA->next;    //B链表中不可能再出现指数相等的结点了，pA可以指向下一个结点
14            pPreA = pPreA->next;
15        } else if (pA->ex > pB->ex){
16            B->next = pB->next;    //B链表中的节点指数更小，拆下来插入到pA之前
17            pB->next = pA;
18            pPreA->next = pB;
19            pPreA = pPreA->next;
20            pB = B->next;      //pB指针指回正确位置
21        } else if (pA->ex < pB->ex) { //其实这里不必写 else if, 写 else 就行
22            pA = pA->next;      //B链表中的节点指数更大，只可能是插入到 pA 后面，应该让 pA
指针往后遍历
23            pPreA = pPreA->next;
24        }
25    }
26    if (pB != NULL){          //多项式A已经到头了，但多项式B还没合并完
27        pPreA->next = B->next;    //把链表B剩余的都抢过来连在A的末尾!
28        B->next = NULL;
29    }
30    return A;
31 }
```

(3) 时间复杂度： $O(n + m)$ ，其中 n 和 m 分别是多项式 A 和 B 的节点数。空间复杂度： $O(1)$ 。

二、(13分) 国产大飞机 **C919** 的研发成功，是我国航空工业发展的重要里程碑，标志着中国具备了自主设计、制造大型喷气式民用客机的能力。随着 **C919** 的成功商业运营，越来越多航空公司开始采用 C919 执飞国内航线。为了方便乘客出行，机场调度中心需要将来自 n 个不同航空公司当天的航班排班表合并成一条**统一排班表**，供旅客查询和调度使用。每个公司的航班排班表用一个**带头结点的递增有序单链表**表示，链表中的每个节点是一个整数，表示航班的起飞时间（单位：分钟，从当天 0 点开始计时）。请你设计一个算法，将 n 个有序排班表合并成一条**仍然递增有序**的排班表。函数原型为：`struct LNode* mergeFlightSchedules(struct LNode* lists[], int n)`

```

1 //单链表节点的定义
2 struct LNode {
3     char airline[100];           // 航空公司名字
4     char flightNumber[20];       // 航班号
5     int val;                     // 航班起飞时间（分钟）
6     struct LNode* next;         // 指向下一个航班
7 } LNode;

```

1) 给出算法的基本设计思想。(4分)

2) 根据设计思想, 采用C或C++语言描述算法, 关键之处给出注释。(9分)

【参考答案】

(1) 可以通过两两归并的策略, 将n个有序单链表合并为一个。

- 首先实现 `merge(LNode *h1, LNode *h2)` 函数, 实现两个有序单链表的归并, 将所有元素归并到 `h1` 并保持有序。
- 在 `mergeFlightSchedules` 函数新建一个空链表, 再通过n次两两归并, 将 `n` 个有序单链表逐步合并成一个有序链表。

(2)

```

1 LNode* merge(LNode* h1, LNode* h2) {
2     LNode* p1 = h1->next; // p1指向h1的第一个有效节点
3     LNode* p2 = h2->next; // p2指向h2的第一个有效节点
4     LNode* pPre = h1;      // pPre指向h1的头结点, 用于插入操作
5
6     while (p1 != NULL && p2 != NULL) {
7         if (p1->val <= p2->val) {
8             pPre = pPre->next; // 向前推进pPre
9             p1 = p1->next;     // 向前推进p1
10        } else {
11            // 将p2插入到p1之前
12            pPre->next = p2;    // 插入p2
13            p2 = p2->next;     // 向前推进p2
14            pPre->next->next = p1; // 将p2->next指向p1
15            pPre = pPre->next;  // 更新pPre为p2
16        }
17    }
18
19    // 如果h2还有剩余节点, 直接连接到h1
20    if (p2 != NULL) {
21        pPre->next = p2;
22    }
23
24    // 归并完成后, h2只保留头结点, h2->next = NULL 可能不是必须的
25    h2->next = NULL;
26
27    return h1; // 返回合并后的h1
28 }
29

```

```

30
31 // 通过n次两两归并，合并n个有序链表
32 LNode* mergeFlightSchedules(struct LNode* lists[], int n) {
33     if (n == 0) return NULL; // 如果没有链表，返回NULL
34
35     LNode* head = (LNode *)malloc(sizeof(LNode));
36     head -> next = NULL; // 整个一个只有头结点的空链表出来
37
38     for (int i = 0; i < n; i++) {
39         merge(head, lists[i]); // 逐一合并
40     }
41     return head; // 返回合并后的最终链表
42 }

```

开动脑瓜子：这个 merge 函数能否用于解决第一题？——当然可以！

三、（思维扩展，可先跳过不作答）：对于第二题，如果不必考虑代码实现，为了让关键字对比次数尽可能地少，你能想到哪些可采用的策略？

思路一：如果仍然采用二路归并的思想，可以利用哈夫曼树，构造一颗最佳二路归并树，减少关键字对比次数

思路二：如果直接采用 n 路归并，可以考虑使用“败者树”减少关键字对比次数

思路三：采用小根堆（优先队列的实现原理）

四、（13分）已知一棵二叉树高度为h，结点总数为n，采用二叉链表存储结构，结点的数据结构定义如下：

```

1 struct TNode {
2     int data;           // 节点编号
3     int bf;             // 平衡因子（左子树高度 - 右子树高度）
4     struct TNode* left; // 左子树
5     struct TNode* right; // 右子树
6 }TNode;

```

请实现一个算法，为二叉树中每个节点计算平衡因子（平衡因子 = 左子树高度 - 右子树高度），并将结果存储在节点的 bf 字段中。函数原型为：void setBalanceFactor(TNode* root)

- 1) 给出算法的基本设计思想。（4分）
- 2) 根据设计思想，采用C或C++语言描述算法，关键之处给出注释。（7分）
- 3) 说明你所设计算法的时间复杂度和空间复杂度。（2分）

【参考答案】

(1) 可以采用后序遍历解决问题：

- 先递归计算左子树高度，再递归计算右子树高度
- 当前节点的平衡因子 $bf = \text{左子树高度} - \text{右子树高度}$
- 当前节点的高度 = $\max(\text{左子树高度}, \text{右子树高度}) + 1$
- 若遍历到空节点，高度计为0

(2)

```
1 //后序遍历左、右子树，通过return给父节点返回高度信息
2 int houXu(TNode * root){
3     if (root == NULL) {
4         return 0; // 空节点高度为0
5     }
6
7     // 递归计算左子树高度
8     int lh = houXu(root->left);
9     // 递归计算右子树高度
10    int rh = houXu(root->right);
11
12    // 设置当前节点的平衡因子
13    root->bf = lh - rh;
14
15    // 返回当前节点的高度
16    if (lh > rh){
17        return lh + 1;
18    } else {
19        return rh + 1;
20    }
21 }
22
23 void setBalanceFactor(TNode* root) {
24     houXu(root);
25 }
```

(3) 时间复杂度= $O(n)$ ，空间复杂度 $O(h)$

五、有一个长度为 n 的升序整数数组 $\text{int } A[n]$ 。试设计一个在时间和空间两方面都尽可能高效的算法，返回值 k 在数组 $A[n]$ 中出现的次数。函数原型为：`int countK(int A[], int n, int k)`

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想，采用C或C++语言描述算法，关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

【参考答案】

(1) 由于数组是升序排列的，可以利用二分查找高效定位 k 的起始位置和结束位置，从而计算 k 出现的次数。首先，二分查找元素 k 的左边界 $left$ 、右边界 $right$ 。最终，元素 k 出现的次数 $= right - left + 1$ 。

(2)

```
1 // 二分查找左边界 (第一个等于k的位置)
2 int findLeft(int A[], int n, int k) {
3     int left = 0, right = n - 1;
4     while (left <= right) {
5         int mid = left + (right - left) / 2;
6         if (k <= A[mid]) {
7             right = mid - 1; // 向左收缩
8         } else {
9             left = mid + 1; // 向右收缩
10        }
11    }
12    // 检查left是否越界或A[left]是否等于k
13    return (left < n && A[left] == k) ? left : -1;
14 }
15
16 // 二分查找右边界 (最后一个等于k的位置)
17 int findRight(int A[], int n, int k) {
18     int left = 0, right = n - 1;
19     while (left <= right) {
20         int mid = left + (right - left) / 2;
21         if (k >= A[mid]) {
22             left = mid + 1; // 向右收缩
23         } else {
24             right = mid - 1; // 向左收缩
25        }
26    }
27    // 检查right是否越界或A[right]是否等于k
28    return (right >= 0 && A[right] == k) ? right : -1;
29 }
30
31 // 主函数: 计算k的出现次数
32 int countK(int A[], int n, int k) {
33     int leftBound = findLeft(A, n, k); // 计算左边界
34     if (leftBound == -1) return 0; // k不存在, 相当于在原数组中总计出现过0次
35     int rightBound = findRight(A, n, k);
36     return rightBound - leftBound + 1; // 根据k值的右边界、左边界差值计算k值出现次数
37 }
```

(3) 时间复杂度 $= O(\log n)$, 空间复杂度 $= O(1)$

六、（本题值20分以上）中国高铁以“安全、快捷、舒适”的优势享誉世界，是中国自主创新的重大成就之一。如今，中国已建成全球最大规模的高速铁路网，实现的诸多城市之间的铁路贯通，向世界展示了“中国速度”和“中国智造”的实力。假设用一个无向图邻接矩阵表示N个城市（城市编号为0~N-1）之间的“直达列车”情况。例如，城市0、城市1之间有直达列车，则两个城市之间的无向边记为1；城市0、城市3之间没有直达列车，则两个城市之间的无向边记为0。如果两个城市之间没有直达列车，就需要换乘。请实现一个算法，计算从城市 `start` 到城市 `end`，最少需要乘坐几趟列车？

城市编号	0	1	2	3	4	5
0	0	1	1	0	0	0
1	1	0	0	1	1	0
2	1	0	0	0	1	1
3	0	1	0	0	0	1
4	0	1	1	0	0	1
5	0	0	1	1	1	0

- (1) 给出邻接矩阵的数据结构定义。
- (2) 给出算法的基本设计思想。
- (3) 根据设计思想，采用C或C++语言描述算法，关键之处给出注释。

注：本题略高于408难度，408算法题分值最多15分。这道题如果按照408标准，至少值20分。

【参考答案】

(1)

```
1 typedef struct {
2     int matrix[N][N];
3     int numCities;
4 } Graph;
```

(2)

本题本质上是无向无权图的最短路径问题，可采用 **BFS（广度优先搜索）** 求解：

- 1. **初始化**：使用队列存储待访问城市，`visited` 数组记录是否已访问，`distance` 数组记录最少乘几趟车。
- 2. **BFS遍历**：
 - 从 `start` 出发，遍历所有直达城市（邻接矩阵中值为 `1` 的节点）。
 - 若未访问过，则更新 `distance` 并入队。
- 3. **终止条件**：到达 `end` 或队列为空（说明不可达）。

(3)

```
1  int minTrains(Graph* g, int start, int end) {
2      if (start == end) return 0;
3
4      bool visited[N] = {false};
5      int distance[N] = {0};
6      int queue[N];          // 用数组模拟队列
7      int front = 0, rear = 0; // 队列头尾指针
8
9      visited[start] = true;
10     queue[rear++] = start; // 入队
11
12     while (front < rear) { // 队列不为空
13         int current = queue[front++]; // 出队
14
15         for (int neighbor = 0; neighbor < g->numCities; neighbor++) {
16             if (g->matrix[current][neighbor] == 1 && !visited[neighbor]) {
17                 distance[neighbor] = distance[current] + 1;
18                 if (neighbor == end) return distance[neighbor]; // 找到终点
19                 visited[neighbor] = true;
20                 queue[rear++] = neighbor; // 入队
21             }
22         }
23     }
24
25     return -1; // 不可达
26 }
```

应用题

七、在一个系统中，用长度为 N 的数组 `data[0...N-1]` 存储两个栈：

- 栈 **S1**：从数组左端（下标 0）向右增长；
- 栈 **S2**：从数组右端（下标 $N-1$ ）向左增长；
- 栈顶指针：`top1` 和 `top2`
- 初始化时：`top1 = 0; top2 = N-1;`

(1) 对于栈 S1、S2，如何判断栈空、栈满、入栈、出栈？

(2) 如何计算共享栈总共有多少个元素？

(3) 如果初始化时，`top1 = -1; top2 = N;` 上面两个小问的答案又是什么？

【参考答案】

(1)

操作	栈 S1 (左→右)	栈 S2 (右→左)
栈空	$\text{top1} == 0$	$\text{top2} == N-1$
栈满	$\text{top1} > \text{top2}$	$\text{top1} > \text{top2}$
入栈	$\text{data}[\text{top1}++] = x;$	$\text{data}[\text{top2}--] = x;$
出栈	$x = \text{data}[\text{--top1}];$	$x = \text{data}[\text{++top2}];$

(2)

共享栈总元素个数 = $\text{top1} + (N - 1 - \text{top2})$;

(3)

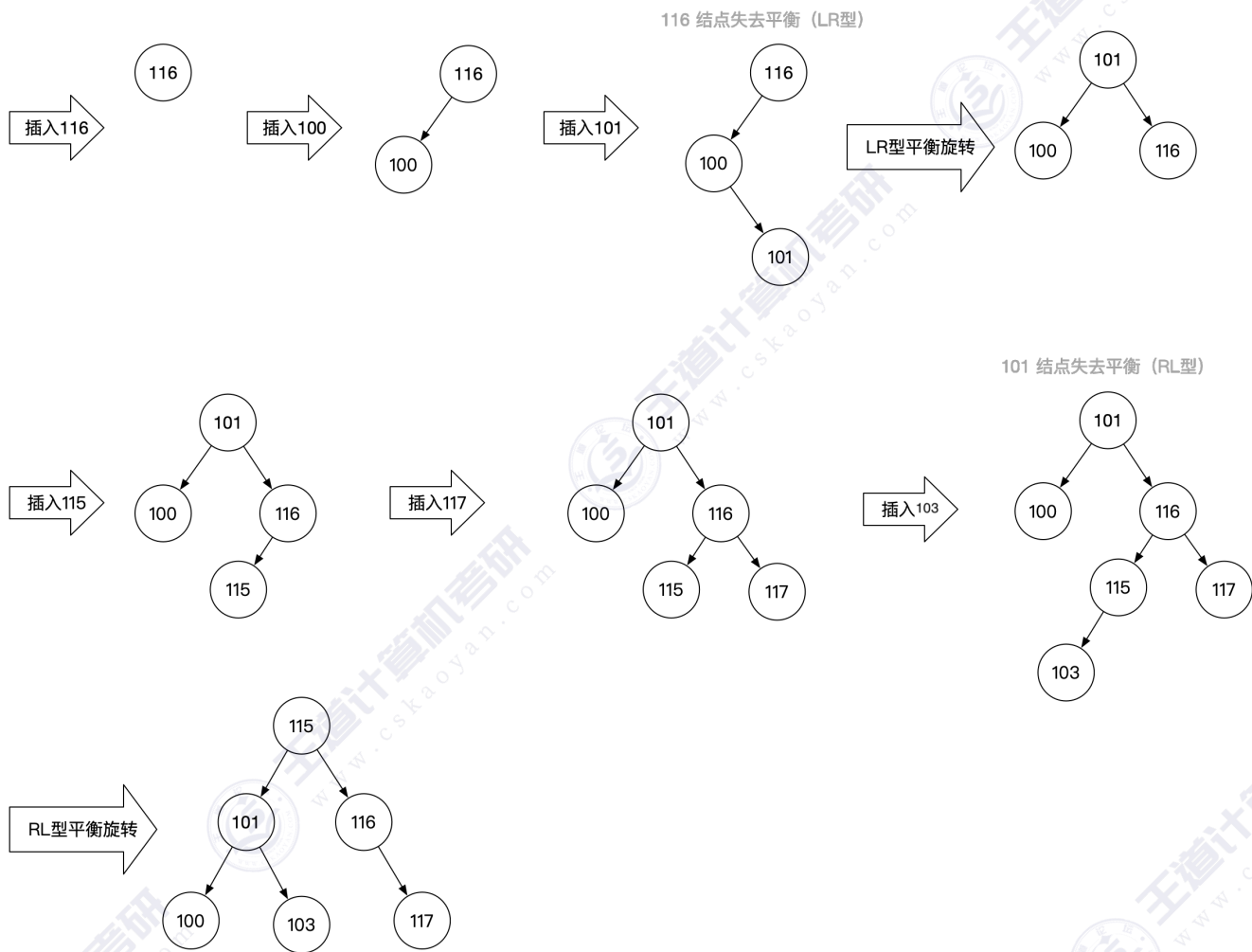
操作	栈 S1 (左→右)	栈 S2 (右→左)
栈空	$\text{top1} == -1$	$\text{top2} == N$
栈满	$\text{top1} + 1 \geq \text{top2}$	$\text{top1} + 1 \geq \text{top2}$
入栈	$\text{data}[\text{++top1}] = x;$	$\text{data}[\text{--top2}] = x;$
出栈	$x = \text{data}[\text{top1--}];$	$x = \text{data}[\text{top2++}];$

八、将关键字序列 {116, 100, 101, 115, 117, 103} 依次插入到初始为空的平衡二叉树 (AVL树)，给出每插入一个关键字后的平衡树，并说明其中可能包含的平衡调整步骤（即：先说明是哪个结点失去平衡，然后说明做了什么平衡处理）；然后分别给出前序、中序和后序遍历该二叉树的输出结果。【中国科学院大学863 2019年】

考点：平衡二叉树

【参考答案】

各关键字的插入过程如下所示：



对该二叉树的遍历结果如下

前序遍历: 115, 101, 100, 103, 116, 117

中序遍历: 100, 101, 103, 115, 116, 117

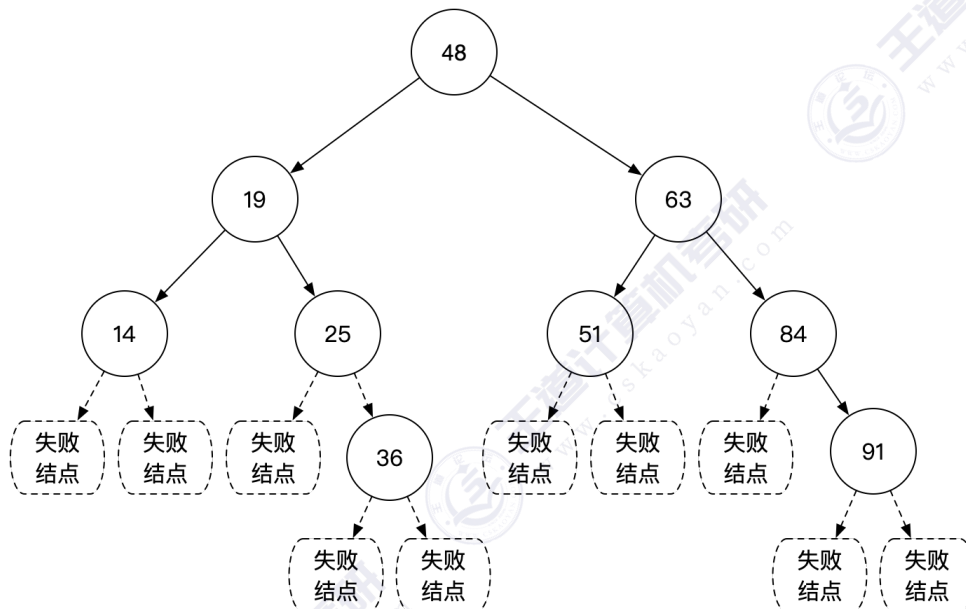
后序遍历: 100, 103, 101, 117, 116, 115

九、设一组有序的记录关键字序列为 (14,19,25,36,48,51,63,84,91)，运用二分法进行查找，请给出二分查找的判断树，以及查找关键字 84 时的比较次数，并计算出查找成功时的平均查找长度。【华中科技大学834 2019年】

考点: 查找算法的分析与应用 (二分查找)

【参考答案】

二分查找的判断树如下:



查找成功时的平均查找长度 $ASL_{成功} = (1 + 2 \times 2 + 3 \times 4 + 4 \times 2) / 9 = 25/9$

十、在一个社交网络平台上，有 n 个用户。平台希望尽可能快速地判断两个用户是否在同一个朋友圈（直接或间接认识）。每个用户最初互不认识，各自形成一个独立的朋友圈。当用户 A 和用户 B 成为朋友时，平台将 A 和 B 的朋友圈合并。请回答：

1. 为了快速判断两个用户是否在同一个朋友圈，可以采用哪种数据结构？请给出数据结构的定义。
2. 当两位用户成为新朋友时，需要对数据结构做什么操作？请给出该操作的代码。
3. 如何查询两个用户是否属于同一个朋友圈？

【参考答案一：用并查集解决问题】

(1) 并查集。数据结构定义参考王道书。

并查集的结构定义如下：

```
#define SIZE 100
int UFSet[SIZE]; //集合元素数组（双亲指针数组）
```

下面是并查集主要运算的实现。

(1) 并查集的初始化操作

```
void Initial(int S[]){ // S 即并查集
    for(int i=0;i<SIZE;i++) //每个自成单元元素集合
        S[i]=-1;
}
```

(2) 并查集的 Find 操作

在并查集 S 中查找并返回包含元素 x 的树的根。

```
int Find(int S[],int x){
    while(S[x]>=0) //循环寻找 x 的根
        x=S[x];
    return x; //根的 S[] 小于 0
}
```

判断两个元素是否属于同一集合，只需分别找到它们的根，再比较根是否相同即可。

(3) 并查集的 Union 操作

求两个不相交子集合的并集。若将两个元素所在的集合合并为一个集合，则需要先找到两个元素的根，再令一棵子集树的根指向另一棵子集树的根。

```
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return; //要求 Root1 与 Root2 是不同的集合
    S[Root2]=Root1; //将根 Root2 连接到另一根 Root1 下面
}
```

Find 操作和 Union 操作的时间复杂度分别为 $O(d)$ 和 $O(1)$ ，其中 d 为树的深度。

(2) 两个用户成为新朋友时候，使用 Union 操作。代码参考王道书。（注意：如果题目没有对效率提出要求，就回答最简易的版本，最不容易出错，书写量也最少。没必要写“小树并入大树”）

(3) Find 操作，检查两个用户是否从属于同一个“集合”，换句话说，是否属于同一个“根节点”。如果是，说明两个用户属于同一个朋友圈

【参考答案二：用无向图解决问题】

(1) 可以用图的邻接矩阵。用二维数组 `guanxi[n][n]` 表示用户关系，其中：

- `guanxi[i][j] = 1`：用户 `i` 和 `j` 是朋友（直接连通）。
- `guanxi[i][j] = 0`：用户 `i` 和 `j` 无直接关系。

(2) 当用户 `A` 和 `B` 成为朋友时：

```
1 adj_matrix[A][B] = 1;
2 adj_matrix[B][A] = 1; // 注意：无向图需对称更新
```

(3) 通过 DFS/BFS 遍历图，检查 `A` 和 `B` 是否连通。如果连通，说明属于同一个朋友圈。算法大致思想如下：

- 从 `A` 出发，DFS/BFS 标记所有能访问到的节点（连通分量）。
- 检查 `B` 是否被标记。

